Meta-Probing Utility Predictor (MPUP): Predicting the Utility of Retrieved Passages for RAG without Fine-Tuning the LLM

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

4th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

5th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

6th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

Abstract—Retrieval-Augmented Generation (RAG) systems rank passages by topical relevance, yet relevance and usefulness are not the same thing. Even a passage that matches a query may fail to help the model answer. Recent utility-aware RAG methods proven to be impractical because existing methods either fine-tune a reranker or retriever's embedding model for every target LLM, or run the target LLM on every candidate passage at selection time, which takes up high cost, memory, and computation. Meta-Probing Utility Predictor (MPUP) is a method to predict LLM-specific retrieval utility without fine-tuning the target model.

MPUP combines 16 inexpensive signals: retriever scores, passage statistics, entity overlap, an NLI check against a hypothetical answer, the change in the LLM's next-token entropy, a cross-encoder score, and a ridge probe on the LLM's hidden state. The regressor, trained on within-query deviations, ranks the passages. On held-out test questions of ten datasets, MPUP reaches a Spearman correlation $r_s = 0.356$ with the true utility, against 0.226 for dense similarity. The gain of +0.13 (57% in relative terms) is highly significant (paired permutation test, $p < 10^{-5}$; Holm-adjusted $p \leq 0.003$ in every dataset; $d_z = 0.45$) and holds for datasets left out of training. With the top passage in the prompt, answer accuracy rises from 0.376 to 0.488. The hidden-state probe drives the gain (development score 0.269 without it and the cross-encoder, 0.346 with both). Trained on Qwen2.5-7B alone, it ranks Mistral-7B passages at 0.313 versus 0.210 for dense similarity. Extraction ran on a laptop within 6.4 GiB of MLX memory.

Index Terms—retrieval-augmented generation, passage utility, utility prediction, passage selection, logit probing

I. Introduction

Large language models (LLMs) are Transformer-based neural networks pre-trained on massive unlabeled text so that they are adapted to specific tasks through prompting rather than task-specific retraining [1], making one model can do reasoning, summarization, and generation tasks that previously require separate architectures [2]. LLMs are used in multiple areas, one of them is LLM for personalized tutoring and automated feedback in education [3]. LLMs can also be used in reshaping professional practice, research workflows, and policy [4].

LLMs often produce hallucinations which are outputs unfaithful to the source or factually incorrect [5], because their knowledge is stored and frozen at training time, making them cannot be corrected without retraining [6]. Retrieval-Augmented Generation (RAG) addresses this problem by retrieving relevant documents from external sources and do generation on that retrieved evidence, so the model draws on up-to-date information instead of only memorized parameters [7]. RAGs have multiple methods to retrieve the documents, one of them is traditional RAG which select documents purely by topical relevance where the contents match the queries, which works worse than RAG systems where the documents are selected by their utility for the model [8].

Selecting documents or passages based on their utility in RAG is done by seeing whether the passage providing it as evidence measurably improves a LLM's ability to generate an accurate answer compared to answering without any evidence [8]. To measure this, two cases need to be compared. In the first, the model answers from its internal memory alone. In the second, the same model answers with a specific retrieved passage available to it. Both answers are scored by a standard correctness metric, and a passage counts as useful only when the second answer is more accurate than the first. Applying the same test to a large pool of candidates leaves only the passages that actually help that model [8].

However, how can a practitioner estimate passage utility under realistic, low-budget conditions? Existing methods are too expensive for practical deployment, because they

require a 32-billion-parameter teacher model to produce supervision [9], completely retraining the retriever's embedding model using complex objectives [10], or running the target LLM twice for every single query [11].

There are cheap alternatives method compared to LLM sampling. Semantic Perplexity (SePer) measures how useful a retrieved passage by observing how much its semantic perplexity changes after the passage is provided, but it still requires running an evaluation model, which adds computation and latency, making it less suitable for real-time passage selection [12]. Another approach proposes a lightweight neural model that predicts passage utility for a target Question Answering (QA) model, but its predictor is trained for a specific QA model, making it difficult to apply to other QA models [13]. Another method is a feature-based predictor combining Query Performance Prediction (QPP), perplexity, readability, and document quality features using linear regression to predict retrieval utility, but this method achieves relatively low accuracy for utility prediction, with Spearman $r_s \approx 0.20$–0.24 for retrieval utility on Natural Questions [14]. Low accuracy and unapplicable solutions show that none of these methods has solved the problem yet.

Several other methods have been explored. There are LURE-RAG which introduces a lightweight LambdaMART reranker trained with a listwise utility loss to optimize context ordering, but relying on complex listwise utility supervision requires generating query and passage combinations, making the labeling cost expensive [15]. Another approach is RRPO, where passages selection treated as a sequential decision process trained via reinforcement learning directly from LLM feedback, which also which takes up big part of the memory and computation [16]. Existing methods either require substantial memory and computational resources, or incur high costs.

Meta-Probing Utility Predictor (MPUP), proposed in this paper, closes the gap. MPUP is a method to predict LLM-specific retrieval utility which means it can select the most useful passages for models without fine-tuning any model. Instead of having the target LLM answer with and without every passage [11] or scoring each passage with a separate evaluation model [12], MPUP evaluates a passage's usefulness by looking at 16 practical clues, such as document statistics, meaning overlap, and agreement with a hypothetical answer. Three of them read the logits of an LLM through one forward pass per passage (logit probing), and one more is a linear probe on the hidden state of the same LLM. Another is the score of a cross-encoder. MPUP runs its LLM-based steps, one short generation per query and two forward passes per passage, with one shared model on a single GPU and does not train any LLM. Section IV reports the measured memory footprint and extraction time.

Contributions of this paper are fourfold. The first is a predictor of the LLM-specific utility of a passage that needs no fine-tuning and no answer generation for each passage: MPUP combines 16 inexpensive signals, among them a ridge probe on the hidden state of the LLM and a cross-encoder score. The second is evidence on where the gain comes from. On the development questions, the model without the probe and the cross-encoder reaches 0.269, the cross-encoder alone lifts it to 0.282, the hidden-state probe alone to 0.344, and both together to 0.346; a smoothed training label lowers every variant. The third is evaluation on test questions never seen during design: MPUP gains 57% in relative r_s over dense similarity ($p < 10^{-5}$, significant in each of ten datasets, also for datasets left out of training) and raises the answer accuracy from 0.376 to 0.488. The fourth is transfer and cost: a predictor trained on Qwen2.5-7B ranks passages for Mistral-7B better than dense similarity and a cross-encoder without any Mistral label, and the extraction runs end to end on a laptop with a peak of 6.4 GiB of MLX memory.

Remaining sections are organized as follows. Section II reviews related work on retrieval-augmented generation, utility-based passage selection, and lightweight utility prediction. Section III describes the proposed Meta-Probing Utility Predictor, covering the 16 features, the predictor architecture, and the model and hardware setup. Section IV presents the experimental setup, datasets, and evaluation metrics, followed by the results and analysis, while Section V concludes the paper and outlines directions for future work.

II. Literature Review

LLM can only use the knowledge it learned during training, and this knowledge cannot be updated without retraining. RAG solves this by first retrieving documents from an external source, then ask the LLM to write its answer based on those documents [7]. In traditional RAG, passages are chosen by relevance, which means how closely their content matches the query.

However, a relevant passage is not always a helpful one. Recent work shows that choosing passages by their utility for the model gives better results than choosing them by relevance alone [8]. Utility is measured by comparing two answers from the same LLM. The first answer is written from the model's own memory. The second is written with the candidate passage. Correctness metric scores both answers, and the passage is useful only if the second answer is more accurate [8].

Utility also depends on what the model already knows. Passage may help one LLM but add nothing for another LLM that already knows the answer. So, utility belongs to a pair of passage and model, not to the passage alone. To estimate it in practice, we can either test the target LLM directly or approximate it with cheaper signals.

Methods that estimate passage utility can be grouped into four types. Utility estimation by large models or utility-trained retrievers is the first type. Zhang et al. [9] distill the utility judgments of a 32-billion-parameter teacher LLM into smaller student models, and Sandhu

et al. [10] retrain the retriever's embedding model with complex objectives so that its ranking follows utility. Both reach utility-aware selection, but at the cost of heavy training, and the former also needs a large teacher model to produce its supervision, which is hard to afford under low-budget conditions. Learned rerankers and selection policies trained from utility feedback form the second. LURE-RAG introduces a lightweight LambdaMART reranker trained with a listwise utility loss [15], while RRPO treats passage selection as a sequential decision process optimized by reinforcement learning from LLM feedback [16]. LURE-RAG depends on expensive query-passage labeling, and RRPO consumes substantial memory and computation.

Utility estimation by directly involving the target LLM is the third. Zhang et al. [11] run the target LLM twice for every query to compare its behavior with and without the passage, and SePer measures how much the semantic perplexity changes once the passage is provided [12]. Methods of this type are closest to the definition of utility as improvement over the model's internal memory, but they add inference cost and latency at selection time, so they are less suitable for real-time use. Lightweight utility predictors form the fourth. Perez-Beltrachini and Lapata [13] propose a lightweight neural model that predicts utility for a target QA model, yet it is trained for one specific model and is difficult to transfer to others. Tian et al. [14] combine Query Performance Prediction (QPP), perplexity, readability, and document quality features in a linear regression, which is cheap but reaches only Spearman $r_s \approx 0.20$–0.24 for retrieval utility on Natural Questions.

MPUP is compared to eight previous studies which were based on six criteria, presented in Table I. C1 is a lightweight model that runs on limited hardware. C2 is no fine-tuning of the LLM or the retriever. C3 is no additional LLM run for every query at selection time. C4 is utility estimation accuracy that is competitive with methods that probe the target LLM directly. C5 is a single predictor that works for several target LLMs without retraining. C6 is training on annotations that already exist in the dataset instead of labels generated by the target LLM.

Table I shows how the studies differ. Methods that reach high accuracy through the target LLM [11], [12] fail the lightweight and per-query cost criteria, while lightweight methods [13], [14] either remain tied to one model or lose accuracy. LURE-RAG [15] is the one study that reports reusing a predictor, trained on the utilities of one LLM, for other LLMs (its fourth research question), but its labels come from running an LLM. MPUP meets C2 by design. MPUP still runs an LLM at selection time (one short generation per query and two forward passes per passage), so it meets C3 only in part. In 4-bit precision, MPUP meets C1, since the extraction ran end to end on a laptop with a peak of 6.4 GiB of MLX memory. For C4 and C5 this paper gives only partial evidence, and C6 is not met because the utility labels come from running the LLM.

TABLE I
Positioning of MPUP to Previous Studies

Method	C1	C2	C3	C4	C5	C6
Zhang et al. [9]	×	×	✓	×	×	✓
Sandhu et al. [10]	×	×	✓	×	×	✓
Zhang et al. [11]	×	✓	×	✓	×	✓
Dai et al. [12] (SePer)	×	✓	×	✓	×	✓
Perez-Beltrachini & Lapata [13]	✓	✓	✓	×	×	✓
Tian et al. [14]	✓	✓	✓	×	×	×
LURE-RAG [15]	✓	✓	✓	×	✓	×
RRPO [16]	×	×	✓	×	×	✓
MPUP (ours)	✓	✓	∼	∼	∼	×

∼: only partly met or partly established (Section IV). C1: met in 4-bit precision, where the extraction of the features and hidden states ran end to end on one Apple M5 laptop (24 GiB of unified memory) with a peak of 6.4 GiB of MLX memory; the cross-encoder was not run there, and 8 GB cards were not tested. In bfloat16 on a GPU the peak is 20.8 to 25.7 GiB. C3: one short generation per query, two forward passes of M per passage (probing and hidden state), and one cross-encoder pass, but no answer generation per passage. C4: MPUP was not compared with a method that probes the target LLM. C5: transfer tested for one pair of LLMs only (Qwen2.5-7B to Mistral-7B, Section IV-F). C6: not met, since the utility labels come from running the LLM.

All partial marks are shown as ∼ in the table.

Utility-based selection is only useful if people can actually run it. Many practitioners have access to only a single GPU and a limited budget, not a training cluster. Methods that need very large models, retriever retraining, or two LLM runs for every query are out of reach for them. Methods that are cheap but tied to one LLM create another problem, because the predictor must be rebuilt whenever the LLM is replaced. Lightweight methods with low accuracy are also not helpful, since they often pick the wrong passages. Every barrier above would be removed by a predictor that is lightweight and does not have to be rebuilt for each new LLM, provided that its ranking is good enough to help. MPUP is designed for this purpose by combining many cheap features into one predictor, and Section IV measures how good its ranking is.

III. Methodology

Fig. 1 shows the MPUP pipeline. For each query, MPUP first asks an LLM M to write one short hypothetical answer. Then, for every candidate passage, it extracts 14 features from five feature families and adds two more, described below (16 features in total). Next, the trained regressor maps these features to a utility score, and an optional calibration step adjusts the scale of the score. Passages are ranked by the score, and the top passages are placed in the prompt of the target LLM. No fine-tuning of the target LLM is ever needed, and the question does not need to be answered twice for every passage.

Passage utility measures whether a passage, given as evidence, improves the accuracy of an LLM compared with answering without any evidence [8]. Let q be query, d

candidate passage, and L target LLM. Utility of d for L on query q is the difference in generation accuracy [8]:

$$u(d; L, q) = \mathrm{Acc}(L(q, d)) - \mathrm{Acc}(L(q, \varnothing)). \tag{1}$$

Where $L(q, \varnothing)$ is the answer of L from its own memory alone, and $L(q, d)$ is the answer of L when it reads d. $\mathrm{Acc}(\cdot)$ is a standard correctness metric. Passage is useful only if reading it improves the model, which means $u > 0$. Zero utility means the passage adds nothing, and a negative value means it makes the answer worse.

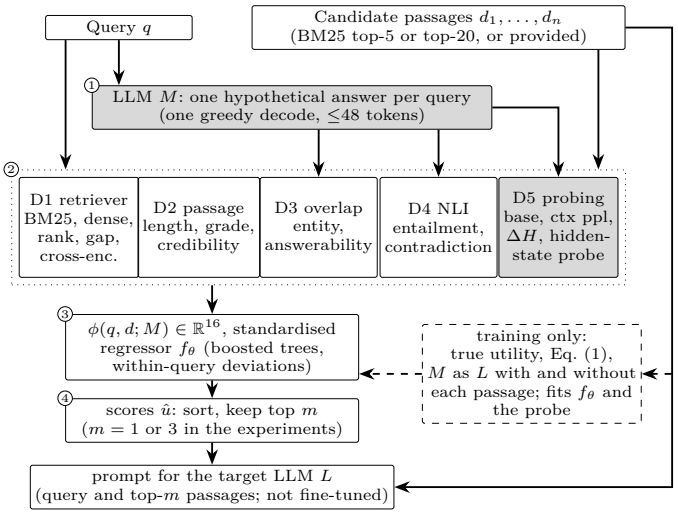


Fig. 1. Overview of MPUP. (1) M writes one hypothetical answer per query. (2) Five feature groups are computed for every passage (arrows into the dotted frame feed all five); D3 and D4 use the answer, D5 probes M. The 16 features include the score of a cross-encoder in D1 and a ridge probe on the hidden state of M in D5. (3) The regressor maps the features to a utility score. (4) The top m passages go, with their text, into the prompt of the target LLM (not fine-tuned). Shaded: uses M, one shared model. Dashed: training only.

If we test a pool of candidate passages $\mathcal{C}$ in this way and keep only the passages that help L, we obtain the gold utilitarian set of L [8]:

$$\widetilde{\mathcal{G}}_q^{(L)} = \{d \in \mathcal{C} \mid u(d; L, q) > 0\}. \tag{2}$$

Acc is the average of relaxed exact match and token-level F1 against the gold answers:

$$\mathrm{Acc}(a) = 0.5 \cdot \mathrm{EM}_r(a) + 0.5 \cdot \mathrm{F1}(a). \tag{3}$$

Relaxed exact match counts an answer as correct when a gold answer appears inside it as a token span. Relaxed matching is used because instruction-tuned LLMs answer in full sentences, and strict exact match would mark most of them wrong, which pushes almost every utility to zero. Values of u lie in $[-1, 1]$.

Building $\widetilde{\mathcal{G}}$ directly needs two LLM runs for every query-passage pair, which is too expensive for real-time selection. MPUP predicts the utility instead, so that passages in $\widetilde{\mathcal{G}}$ are ranked first:

$$\hat{u}(d; q) = f_\theta(\phi(q, d; M)), \tag{4}$$

where $\phi(q, d; M) \in \mathbb{R}^{16}$ is the feature vector and f_θ is the predictor. M is the LLM used for the LLM-based steps: the hypothetical answer, logit probing, and the hidden-state pass. Either the target LLM L or a fixed model can serve as M. In our experiments, M is Qwen2.5-7B-Instruct. Predictor f_θ is trained once and is not retrained when L changes.

Table II lists all 16 features. D1 describes how the retriever scored the passage: its BM25 score (divided by the highest score of the query), its dense similarity, its rank, and its score gap to the next passage. D2 describes the passage itself: token length, Flesch–Kincaid grade, and a source credibility score. D3 measures meaning overlap. Entity overlap is the fraction of query entities, found by spaCy, that also appear in the passage. Answerability is the fraction of content words of the hypothetical answer that appear in the passage. D4 checks faithfulness with a DeBERTa-v3-small NLI model. Here, the passage is the premise and the hypothetical answer is the hypothesis, and we keep the entailment and contradiction probabilities. D5 probes the LLM M. Next-token entropy is computed at the end of two prompts: the query alone (H_{base}) and the passage followed by the query (H_{ctx}). D5 features are the two perplexities $\exp(H)$ and the difference $\Delta H = H_{\mathrm{base}} - H_{\mathrm{ctx}}$. Generated once per query, the hypothetical answer is shared by D3 and D4.

TABLE II
Features of MPUP

Group	Features
D1 Retriever	BM25 score, dense similarity, rank, score gap, cross-encoder score
D2 Document quality	Token length, Flesch–Kincaid grade, source credibility
D3 Semantic overlap	Entity overlap, HyDE answerability
D4 Faithfulness	Entailment probability, contradiction probability
D5 LLM probing	Base perplexity, context perplexity, ΔH, hidden-state probe score

a) Hidden-state probe and cross-encoder: D5 summarises only the next-token distribution of M in three numbers. MPUP also reads the hidden state of M. For every pair, M receives the prompt that defines the true utility in Eq. (1) (passage, then question), and the hidden state $h(q, d) \in \mathbb{R}^{3584}$ of layer 21 of 28 at the last prompt token is kept. Gold answers are never part of the input. Hidden states are centred within the query and standardised, and a ridge regression maps them to the within-query deviation of the training label, with the regularisation strength set to ten times the number of training pairs. Probe output becomes one more D5 feature. Inside a training set the probe scores are out-of-fold (five folds over queries), so the regressor never sees a probe score fitted on the same pair. MPUP also adds the score

of a cross-encoder reranker (bge-reranker-v2-m3, 568M parameters, initialised from BGE-M3 [17]) as a fifth D1 feature, which gives 16 features. Neither addition fine-tunes M or L. Layer and regularisation strength were fixed on a pilot sample of 100 questions per dataset, whose questions are excluded from every test set below.

Standardized features are passed to a regressor, a gradient-boosted regression tree model trained on within-query deviations: the mean of the query is subtracted from every feature and from the label before fitting, and at inference the candidates of the query being ranked form that group. Much of the raw utility is an offset per query (a question that the LLM already answers makes every passage useless), which no reranker can use. Centred models spend no capacity on it, and their scores mean "better or worse than the average candidate". The size of the ensemble is chosen on the development questions among the variants described in Section IV-D.

The predictor is trained on the true utility (1) of the training datasets, computed with the same LLM M acting as the source LLM. Generated once per query, the no-context answer is reused for all passages of that query.

Optionally, $\hat{u}$ can be calibrated when labels from the target LLM are available for k passages. Coefficients α and β are fitted by linear regression on these k examples, and the calibrated score is

$$\hat{u}' = \alpha \cdot \hat{u} + \beta. \tag{5}$$

Calibration changes the scale of $\hat{u}$, which matters for the threshold τ. Rankings do not change as long as $\alpha > 0$, because the order of the scores stays the same. Calibration is not used in the main results.

The LLM-based steps, hypothetical answer generation, logit probing and the hidden-state pass share one model M: Qwen2.5-7B-Instruct, run in bfloat16 on a single NVIDIA RTX 6000 Ada GPU (48 GB). Sharing one copy avoids loading a second 7B model, and full-precision weights mean that no quantization error enters the probing and generation steps. Alternatively, the implementation can load M in 4-bit NF4 with double quantization to reduce memory; we report peak memory for both settings in Section IV. Also available is an Apple Silicon backend (MLX). Both the NLI model and the regressor are small and run without quantization.

At inference time, MPUP makes one short greedy decode per query (at most 48 tokens) for the hypothetical answer. For each passage it makes one probing pass, one NLI pass, one pass of the cross-encoder, and one forward pass of M for the hidden state (the implementation does not merge the latter with the probing pass), then applies the probe and the regressor. Passages are sorted by score, and the top m ($m = 5$ by default, and $m = 1$ and 3 in the answer experiments) are written as numbered context before the query. Dropping passages with a score $\leq \tau$ is also supported. With the selected predictor, which is trained on within-query deviations, the score is relative to the other candidates of the same query, so $\tau = 0$ would drop roughly the less useful half of the candidates rather than the passages with negative utility. No threshold is therefore applied. Answer accuracy with the top one and top three passages is evaluated in Sections IV-E and IV-F.

IV. Experimental Setup

A. Datasets

Table III lists the eleven datasets with gold answers used in the experiments, so the utility of every query-passage pair is computed with Eq. (1). HotpotQA, 2WikiMultiHopQA, and MuSiQue use all the paragraphs they provide.

B. Evaluation protocol

All metrics are computed within each query and then averaged over queries, because a reranker can only order the passages of one query. Queries are used when they have at least two passages and their true utilities are not all equal (1,641 of the 2,094 test queries). Spearman correlation r_s between the predicted score and the true utility serves as the main metric. Intervals are 95% percentile bootstrap intervals over queries (2,000 resamples), and differences between methods are paired on the same queries. Splits are always by query, never by row.

Three kinds of numbers appear and should not be confused. The effect is the paired difference Δr_s between two methods (or ΔAcc for answer accuracy): larger is better, and zero means no gain. The evidence is the two-sided p-value of H_0: $\Delta = 0$, reported next to a 95% interval: smaller is better, and $p < 0.05$ (after Holm correction when several datasets are tested) means that a difference of zero is unlikely, as does an interval that excludes zero. The size of the effect is the standardised $d_z = \bar{\Delta}/s_\Delta$ over queries: larger is better, with about 0.2 small and 0.5 medium. Even a tiny p can accompany a tiny effect when there are many queries, so all three are given.

Because the predictor is chosen from data, the variants are compared by leave-one-dataset-out on development questions only, and the chosen variant is scored once on test questions that were never used for any choice (Section IV-D).

C. Setup

Qwen2.5-7B-Instruct, run in bfloat16 on one NVIDIA RTX 6000 Ada GPU, serves as LLM M. Model M writes the hypothetical answer, produces the probing features, and acts as the source LLM that defines the true utility (greedy decoding, at most 32 new tokens, relaxed exact match). Multilingual-E5-large provides the dense feature and the dense baseline. Baselines are BM25 and dense cosine similarity used alone as ranking scores.

TABLE III
Datasets used for MPUP

Dataset	Source and split (size)	Candidate passages per question	Utility label
Natural Questions, open [18]	FlashRAG nq, test (3,610)	Top twenty by BM25	Eq. (1)
TriviaQA, unfiltered [19]	FlashRAG triviaqa, test (11,313)	Top twenty by BM25	Eq. (1)
WebQuestions [20]	FlashRAG web_questions, test (2,032)	Top twenty by BM25	Eq. (1)
EntityQuestions [21]	Authors' release, test (22,075)	Top twenty by BM25	Eq. (1)
PopQA [22]	FlashRAG popqa, test (14,267; 12,187 after duplicates)	Top twenty by BM25	Eq. (1)
SQuAD [23]	FlashRAG squad, dev (10,570)	Top twenty by BM25	Eq. (1)
AmbigQA [24]	FlashRAG ambig_qa, dev (2,002)	Top twenty by BM25	Eq. (1)
Bamboogle [25]	FlashRAG bamboogle, test (125)	Top twenty by BM25	Eq. (1)
HotpotQA, distractor [26]	hotpotqa/hotpot_qa, validation (7,405; 6,947 without yes/no)	Provided: ten paragraphs	Eq. (1)
2WikiMultiHopQA [27]	FlashRAG 2wikimultihopqa, dev (12,576; 11,268 without yes/no)	Provided: ten paragraphs	Eq. (1)
MuSiQue [28]	dgslibisey/MuSiQue, validation (2,417)	Provided: up to twenty paragraphs	Eq. (1)

D. Study design

The study samples 700 questions from each of the eleven datasets with gold answers (all 125 of Bamboogle), after removing yes/no questions (HotpotQA, 2WikiMultiHopQA) and questions whose text occurs more than once with different answers (PopQA), and takes the top twenty BM25 passages over the English Wikipedia corpus of 21,015,324 passages of 100 words [29] when a dataset provides none. HotpotQA, 2WikiMultiHopQA, and MuSiQue keep their provided paragraphs. Together, the study has 7,125 questions and 128,467 query–passage pairs, every pair labelled with Eq. (1) by the same M and settings as above.

Every question is assigned once, by a hash of its identifier, to a development part (70%) or a test part (30%). Two pilot samples of 100 questions per dataset (five BM25 candidates, or the provided paragraphs), drawn earlier with different seeds, fixed the layer and the regularisation strength of the probe and the grid of variants. Questions that appeared in either pilot sample always go to development, so no test question was seen while MPUP was designed. Bamboogle has almost no question left for testing and is excluded from the test averages. Development holds 5,031 questions and test 2,094, of which 1,641 are rankable. On development only, 24 variants are compared by leave-one-dataset-out: three tree ensembles (100 trees of depth 2 and 300 of depth 3 with learning rate 0.05, 500 of depth 4 with learning rate 0.03, all centred, subsample 0.8), each with or without the cross-encoder feature, with or without the hidden-state probe, and trained on the true utility or on its average with the change in log-probability of the gold answer. Fitted on the development questions, the best variant is then scored once on the test questions. MPUP is the chosen variant (500 trees of depth 4, true utility, cross-encoder score and probe). Two settings are reported: trained on the development questions of all datasets, and trained on the development questions of the other datasets only, so that neither the test questions nor their dataset were seen in training. Used alone, the cross-encoder is an additional baseline.

Mistral-7B-Instruct-v0.3, from a different model family, serves as a second target LLM. Every test question of the larger study and a hash-selected 40% of its development questions (4,099 questions, 73,885 pairs) are labelled again with Eq. (1), now with Mistral answering, and the 14 features are recomputed with Mistral as M (hypothetical answer and logit probing), together with its hidden states at layer 24 of 32. Test questions are the same as above; the variant chosen on the Qwen development questions is

reused without new selection, and every score is against the utility for Mistral. Three uses are compared: MPUP fitted on Mistral's labels with Mistral's features and hidden states; MPUP fitted on Mistral's labels but with every LLM-side feature and the probe computed by Qwen, so that the model that is read differs from the target LLM; and the MPUP predictor of the Qwen study applied unchanged, without any Mistral label.

E. Results

On the development questions, the hidden-state probe is the addition that matters. Under leave-one-dataset-out, the model without the probe and the cross-encoder reaches $r_s = 0.269$ (dense similarity 0.227, cross-encoder 0.257). Adding the cross-encoder feature gives 0.282, adding the probe gives 0.344, and adding both with the largest ensemble gives 0.346, the variant kept for testing. Averaging the label with the change in log-probability of the gold answer lowered every variant (best 0.320).

Table V shows the single evaluation on the test questions. Trained on the development questions of all datasets, MPUP reaches $r_s = 0.356$, against 0.260 for the cross-encoder, 0.226 for dense similarity, and 0.106 for BM25. Paired over the 1,641 test queries, MPUP gains +0.130 [+0.116, +0.144] over dense similarity ($d_z = 0.45$) and +0.095 [+0.083, +0.108] over the cross-encoder ($d_z = 0.37$); every sign-flip permutation test gives $p < 10^{-5}$. Table VI lists every dataset, and Fig. 2 shows the gain over dense similarity in each of them. Gains hold on each of the ten test datasets: the Holm-adjusted p is at most 0.003 against dense similarity and at most 0.012 against the cross-encoder in every dataset, in both settings. Trained without the dataset being tested, MPUP still reaches 0.351, and its gains over dense similarity (+0.124 [+0.111, +0.138]) and the cross-encoder (+0.090 [+0.078, +0.102]) barely change. Top-1 accuracy, the share of queries whose highest-scored passage has the highest true utility, rises from 0.459 with dense similarity to 0.618.

a) Size of the gain: Against dense similarity, MPUP raises the macro-average r_s from 0.226 to 0.356, which is +0.130, or 57% in relative terms; Table IV gives the test against every baseline. The evidence is strong: the sign-flip permutation test gives $p < 10^{-5}$ over the 1,641 test queries (the smallest value that 200,000 flips can resolve), and the Holm-adjusted p is at most 0.003 in each of the ten datasets. The standardised effect, $d_z = 0.45$, is close to a medium effect. For selection, the passage with the highest true utility is ranked first for 61.8% of the queries instead of 45.9%, and the answer accuracy with one passage rises from 0.376 to 0.488, which closes about 40% of the distance between dense similarity and the oracle (0.654). MPUP has the higher r_s than dense similarity on 72% of the test queries and is ahead in all ten datasets (an exact sign test over the datasets gives $p = 0.002$). For scale, Tian et al. [14] report Spearman correlations of 0.19 to 0.26 for predicting retrieval utility on Natural Questions, so an absolute r_s of 0.356 is not low, although their task (predicting the utility of a retrieved set across queries) differs from ranking the passages within a query. The gain is clear and consistent, but it is not a leap: the absolute r_s of 0.356 is moderate, a gap of about 0.17 in answer accuracy to the oracle remains, and with three passages in the prompt the gain over dense similarity shrinks to +0.050.

TABLE IV
Significance of the gain of MPUP over each baseline on the test questions of the larger study (paired over queries). r_s: macro-average within-query Spearman correlation with the true utility (larger is better). Δr_s: MPUP minus the baseline, with its 95% bootstrap interval (larger is better), and the same difference relative to the baseline. p: two-sided paired sign-flip permutation test with 200,000 flips (smaller is better; H_0: the mean difference is zero; the smallest value it can resolve is about 5×10^{-6}). d_z: standardised effect (larger is better).

Baseline	r_s base	r_s MPUP	Δr_s [95% CI]	Relative	p	d_z
Trained on the development questions of all datasets (1,641 test queries)						
Dense similarity	0.226	0.356	+0.130 [+0.116, +0.144]	+57%	$< 10^{-5}$	0.45
Cross-encoder	0.260	0.356	+0.095 [+0.083, +0.108]	+37%	$< 10^{-5}$	0.37
BM25	0.106	0.356	+0.249 [+0.231, +0.266]	+235%	$< 10^{-5}$	0.67
Trained without the dataset being tested						
Dense similarity	0.226	0.351	+0.124 [+0.111, +0.138]	+55%	$< 10^{-5}$	0.44
Cross-encoder	0.260	0.351	+0.090 [+0.078, +0.102]	+35%	$< 10^{-5}$	0.35
BM25	0.106	0.351	+0.243 [+0.226, +0.261]	+229%	$< 10^{-5}$	0.67

TABLE V
Larger study, single evaluation on 1,641 rankable test questions (ten datasets, twenty candidates per retrieved question). Macro within-query Spearman r_s over datasets (larger is better); NDCG@10 and top-1 over queries. Upper block: trained on the development questions of all datasets. Lower block: trained on the development questions of the other datasets only. Baselines need no training.

Method	r_s	NDCG@10	Top-1
MPUP	0.356	0.780	0.618
MPUP, dataset unseen	0.351	0.772	0.598
Cross-encoder	0.260	0.697	0.490
Dense similarity	0.226	0.659	0.459
BM25	0.106	0.557	0.356

b) Answer accuracy: Ranking quality matters only if the selected passages help the answer. For every test question of the larger study (2,093 questions in ten datasets, rankable or not), each method places its top one or top three passages into the prompt of Eq. (1) (one block per passage), and M answers. Table VII gives Acc (3), macro-averaged over datasets. With one passage, MPUP reaches 0.488, against 0.376 with dense similarity, 0.400 with the cross-encoder, and 0.246 without any passage; the paired gain over dense similarity is +0.111 [+0.094, +0.128] and over the cross-encoder +0.088 [+0.073, +0.103] ($p < 10^{-4}$ for both). MPUP is ahead of dense similarity in all ten datasets, and after Holm's correction the difference is significant in five. With three passages, the methods move closer, because a good passage is more often among any three: MPUP reaches 0.494, ahead of dense similarity by +0.050 [+0.035, +0.065] and of the cross-encoder by

TABLE VI

Larger study, per test dataset (trained on the development questions of all datasets; rankable test questions). Within-query Spearman r_s with the true utility for each method. Δr_s: paired difference MPUP minus dense similarity (larger is better). p_{H}: Holm-adjusted two-sided p-value of a paired sign-flip permutation test over queries (smaller is better; H_0: the mean difference is zero), against dense similarity and against the cross-encoder, adjusted over the ten datasets. †Over all test queries together, not an average of the rows; the pooled test is one test and needs no adjustment.

Test dataset	Queries	MPUP	Cross-enc.	Dense	BM25	Δr_s vs. dense	p_{H} (dense)	p_{H} (cross-enc.)
2WikiMultiHopQA	170	0.25	0.18	0.16	0.06	+0.088	0.002	0.011
AmbigQA	147	0.40	0.32	0.31	0.15	+0.095	<0.001	<0.001
EntityQuestions	182	0.43	0.35	0.31	0.22	+0.120	<0.001	<0.001
HotpotQA	197	0.46	0.36	0.32	0.15	+0.138	<0.001	<0.001
MuSiQue	163	0.31	0.22	0.12	0.03	+0.186	<0.001	<0.001
NQ	151	0.34	0.22	0.23	0.04	+0.113	<0.001	<0.001
PopQA	134	0.39	0.30	0.25	0.15	+0.138	<0.001	<0.001
SQuAD	171	0.30	0.25	0.19	0.12	+0.102	<0.001	0.007
TriviaQA	185	0.40	0.28	0.25	0.17	+0.151	<0.001	<0.001
WebQuestions	141	0.28	0.11	0.11	-0.01	+0.166	<0.001	<0.001
Macro mean	1641	0.356	0.260	0.226	0.106	+0.130†	<0.001†	<0.001†

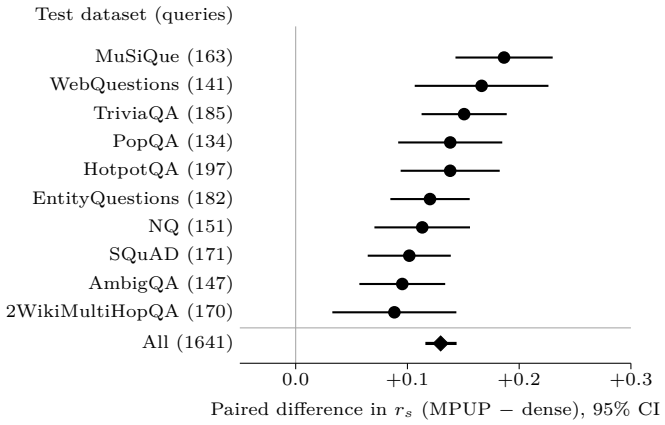


Fig. 2. Paired difference in within-query Spearman r_s between MPUP and dense similarity on each test dataset of the larger study (trained on the development questions of all datasets; Qwen2.5-7B-Instruct, bfloat16, twenty candidates per retrieved question). Filled circles with lines: the mean of the paired per-query differences with its 95% t interval; every interval excludes zero. Last row (diamond): the difference over all 1,641 rankable test queries together, with its 95% bootstrap interval.

+0.019 [+0.006, +0.033]. Selecting by the true utility (oracle) gives 0.654 with one passage, so a gap of about 0.17 remains.

TABLE VII

Answer accuracy (Acc, Eq. (3)) of M with the top-m passages chosen by each method, 2,093 test questions of the larger study, macro mean over ten datasets. Without any passage, Acc is 0.246.

Method	$m = 1$	$m = 3$
MPUP	0.488	0.494
Cross-encoder	0.400	0.475
Dense similarity	0.376	0.444
BM25	0.297	0.370
Random	0.196	0.266
Oracle (true utility)	0.654	0.552

F. Second target LLM

Utility depends on the LLM. On the same 73,885 pairs, 49% of Mistral's utilities are nonzero against 36% of Qwen's, and the two utilities correlate at 0.60 within queries (centred Pearson). Table VIII scores every method against Mistral's utility on 1,714 rankable test questions. Fitted on 2,005 development questions with Mistral as M, MPUP reaches 0.373, against 0.243 for the cross-encoder and 0.210 for dense similarity; paired gains are +0.161 [+0.147, +0.176] over dense similarity ($d_z = 0.53$) and +0.128 [+0.114, +0.142] over the cross-encoder ($d_z = 0.44$), $p < 10^{-5}$, and the Holm-adjusted p is at most 0.004 against both baselines in every one of the ten datasets. Even with fewer development questions than in the Qwen study, MPUP gains at least as much, so its gain is not specific to Qwen.

Reading Qwen instead of Mistral costs part of the gain but keeps most of it. With Mistral's labels but Qwen's features and probe, MPUP reaches 0.322, which is 0.050 [0.040, 0.061] below reading Mistral itself and still +0.111 [+0.098, +0.123] above dense similarity and +0.078 [+0.066, +0.089] above the cross-encoder. Without a single Mistral label, the Qwen predictor applied unchanged reaches 0.313, above dense similarity by +0.101 [+0.088, +0.114] and above the cross-encoder by +0.068 [+0.056, +0.080] ($p < 10^{-5}$ for both); it is ahead of dense similarity in all ten datasets (Holm $p \leq 0.036$) and ahead of the cross-encoder in all ten, significantly so in seven. Qwen's own utility, which requires running Qwen with and without each passage and knowing the gold answer, ranks at 0.512, an upper reference for what one LLM can say about another. Fitted to labels shuffled within each query, the probe falls to 0.013, so its signal comes from the pairing of passages and utilities rather than from the hidden states alone.

With Mistral answering and the top passage in the prompt, MPUP (fitted with Mistral as M) reaches an answer accuracy of 0.479, against 0.380 with dense simi-

larity, 0.391 with the cross-encoder, and 0.280 without any passage (oracle 0.632). Paired gains are +0.098 [+0.084, +0.112] over dense similarity and +0.086 [+0.072, +0.100] over the cross-encoder; MPUP is ahead of dense similarity in all ten datasets and significantly so after Holm's correction in six. With three passages, MPUP reaches 0.475 against 0.424 with dense similarity and 0.450 with the cross-encoder.

TABLE VIII
Second target LLM: within-query Spearman r_s, NDCG@10, and top-1 accuracy against the utility for Mistral-7B-Instruct, 1,714 rankable test questions, macro mean over ten datasets. "Read" is the LLM whose features and hidden states the predictor uses; "Labels" are the utilities it is fitted to.

Method	Read / Labels	r_s	NDCG	Top-1
MPUP	Mistral / Mistral	0.373	0.778	0.544
MPUP	Qwen / Mistral	0.322	0.757	0.536
MPUP	Qwen / Qwen	0.313	0.755	0.537
Probe alone	Mistral / Mistral	0.367	0.765	0.533
Cross-encoder	–	0.243	0.688	0.397
Dense similarity	–	0.210	0.660	0.380
BM25	–	0.095	0.571	0.285
Utility for Qwen	–	0.512	0.787	0.613

G. Cost

Table IX gives the peak GPU memory of the full pipeline (generation, labelling, probing, and NLI), measured on the datasets with the longest contexts (40 questions from MuSiQue and HotpotQA, batch size 32). Labelling and feature extraction for the pilot sample (twelve datasets, 9,554 query–passage pairs, 7,997 of them labelled with the true utility) took 27 minutes in bfloat16 on one GPU, about 0.17 s per pair. Evaluation itself runs on the CPU. Over the whole extraction run for a second pilot sample (twelve datasets), the peak in bfloat16 was 22.3 GiB. For the larger study (128,467 pairs, up to twenty candidates per question), labelling and the 14 features took 5.7 hours, with a peak of 25.7 GiB in bfloat16. Hidden states and cross-encoder scores for MPUP took a further 78 minutes (about 36 ms per pair) in a separate pass, which also scores the gold-answer log-probability used only during development. For Mistral (73,885 pairs), labelling and the 14 features took 3.8 hours with a peak of 27.2 GiB, and the hidden states a further 45 minutes. Reading the hidden state from the same forward pass as D5 would avoid that second pass, but was not implemented. The behaviour on a card or laptop with 8 GB of memory was not measured. On one Apple M5 laptop (24 GiB of unified memory, MLX), the same extraction for the first sample, with Qwen2.5-7B-Instruct in 4-bit precision and two layers of hidden states per pair, took 2.9 hours and peaked at 6.4 GiB of MLX memory over the whole run; the embedding and NLI models run outside MLX and are not counted.

TABLE IX
Peak memory of the pipeline with Qwen2.5-7B-Instruct: GPU memory, and MLX memory on the laptop (where the cross-encoder was not run).

Weights	Peak memory (GiB)
bfloat16	20.8
4-bit NF4 (GPU, small sample)	7.3
4-bit MLX (Apple M5, full run)	6.4

H. Limitations

Results use one source LLM per run. The study uses up to 700 questions per dataset, with its test questions drawn from the same datasets as its training questions; even in the setting that holds a dataset out, its development questions helped choose the variant. Intervals therefore reflect the variation among questions, not among models. Bamboogle is too small to keep test questions apart from the pilot samples and is missing from the test results of the larger study. Transfer between LLMs is tested with one pair only (Qwen2.5-7B to Mistral-7B, both 7B instruction-tuned models, in one direction), on the same questions; reading the target LLM itself was 0.05 better than reading Qwen, and other sizes and families may differ. Answer accuracy was measured only in the larger study, with the LLM that defines the labels answering, greedy decoding, and $m \in \{1, 3\}$. On the laptop, only the extraction of the features and the hidden states was run, not the cross-encoder. Extraction time above includes the labelling with the true utility; the latency of selection alone (one generation per query, then probing, NLI, and the regressor per passage) was not measured. The probing prompt (passage, then query) is truncated at 512 tokens from the right, so for 35 of the 9,554 pairs of the pilot sample (0.37%) the query is cut off and their D5 features are computed without it. Candidate sets hold at most twenty passages, which limits how much any reranker can change.

V. Conclusion

MPUP is a predictor of the utility of a retrieved passage for an LLM that combines 16 inexpensive signals, among them a probe on the hidden state of the LLM and a cross-encoder score, and does not fine-tune the target model. In a larger study with twenty candidates per question, MPUP ranks held-out questions clearly better than dense similarity and a cross-encoder reranker in every dataset (r_s 0.356 against 0.226 and 0.260), also on datasets left out of training, and with the top passage raises answer accuracy from 0.376 to 0.488. The gain over dense similarity is +0.130 in r_s (57% in relative terms, $p < 10^{-5}$, $d_z = 0.45$, close to a medium effect), clear and consistent in every dataset but not a leap: the absolute r_s is moderate, and a gap of about 0.17 in answer accuracy to the oracle remains. The hidden-state probe is the main source of the gain: on the development questions, the score rises from 0.269

without the probe and the cross-encoder to 0.346. For a second target LLM, Mistral-7B, MPUP reaches 0.373 against 0.210 for dense similarity, and the Qwen predictor applied without Mistral labels keeps 0.313. Fitting on one GPU, the pipeline peaks at 21–26 GiB in bfloat16 and 6.4 GiB in 4-bit precision on a laptop. Future work should test transfer across more LLMs, sizes, and directions, measure the latency of selection alone, and evaluate on datasets that are not in the pool.

References

[1] B. Min, H. Ross, E. Sulem, A. P. B. Veyseh, T. H. Nguyen, O. Sainz, E. Agirre, I. Heintz, and D. Roth, "Recent advances in natural language processing via large pre-trained language models: A survey," ACM Comput. Surv., vol. 56, no. 2, pp. 1–40, Sep. 2023.

[2] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang et al., "A survey on evaluation of large language models," ACM Trans. Intell. Syst. Technol., vol. 15, no. 3, pp. 1–45, Mar. 2024.

[3] E. Kasneci, K. Sessler, S. Küchemann, M. Bannert, D. Dementieva et al., "ChatGPT for good? On opportunities and challenges of large language models for education," Learn. Individ. Differ., vol. 103, p. 102274, Apr. 2023.

[4] Y. K. Dwivedi, N. Kshetri, L. Hughes, E. L. Slade, A. Jeyaraj et al., "Opinion paper: "So what if ChatGPT wrote it?" Multidisciplinary perspectives on opportunities, challenges and implications of generative conversational AI for research, practice and policy," Int. J. Inf. Manage., vol. 71, p. 102642, Aug. 2023.

[5] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," ACM Comput. Surv., vol. 55, no. 12, pp. 1–38, 2023.

[6] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," ACM Trans. Inf. Syst., vol. 43, no. 2, pp. 1–55, 2025.

[7] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, "A survey on RAG meeting LLMs: Towards retrieval-augmented large language models," in Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining (KDD), 2024, pp. 6491–6501.

[8] H. Zhang, M. Tang, K. Bi, and J. Guo, "Beyond relevance: Utility-centric retrieval in the LLM era," in Proc. 49th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval (SIGIR), Tutorial, 2026.

[9] H. Zhang, K. Bi, J. Guo, J. Zhang, S. Wang, D. Yin, and X. Cheng, "Distilling a small utility-based passage selector to enhance retrieval-augmented generation," in Proc. Annu. Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval Asia Pacific Region (SIGIR-AP), 2025.

[10] R. Sandhu, D. Mu, C. Chang, M. S. Tasjid, H. Rai, M. Volkovs, and G. Wu, "Aligning dense retrievers with LLM utility via distillation," arXiv preprint arXiv:2604.22722, 2026.

[11] H. Zhang, K. Bi, J. Guo, J. Zhang, S. Wang, D. Yin, and X. Cheng, "LLM-specific utility for retrieval-augmented generation," in Proc. 34th ACM Int. Conf. Inf. Knowl. Manage. (CIKM), 2026.

[12] L. Dai, Y. Xu, J. Ye, H. Liu, and H. Xiong, "SePer: Measure retrieval utility through the lens of semantic perplexity reduction," in Proc. Int. Conf. Learn. Representations (ICLR), 2025, spotlight.

[13] L. Perez-Beltrachini and M. Lapata, "Uncertainty quantification in retrieval augmented question answering," Trans. Mach. Learn. Res., 2025. [Online]. Available: https://openreview.net/forum?id=JLkgI0h7wy

[14] F. Tian, D. Ganguly, and C. Macdonald, "Predicting retrieval utility and answer quality in retrieval-augmented generation," arXiv preprint arXiv:2601.14546, 2026.

[15] M. Chandra, D. Ganguly, and I. Ounis, "LURE-RAG: Lightweight utility-driven reranking for efficient RAG," arXiv preprint arXiv:2601.19535, 2026.

[16] Y. Wu, X. Shen, F. Wang, C. Zhou, Z. Wu, X. Dai, and R. Xia, "Optimizing RAG rerankers with LLM feedback via reinforcement learning," arXiv preprint arXiv:2604.02091, 2026.

[17] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," arXiv preprint arXiv:2402.03216, 2024.

[18] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti et al., "Natural questions: A benchmark for question answering research," Trans. Assoc. Comput. Linguist., vol. 7, pp. 453–466, 2019.

[19] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer, "TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension," in Proc. 55th Annu. Meeting Assoc. Comput. Linguist. (ACL), 2017, pp. 1601–1611.

[20] J. Berant, A. Chou, R. Frostig, and P. Liang, "Semantic parsing on Freebase from question-answer pairs," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2013, pp. 1533–1544. [Online]. Available: https://aclanthology.org/D13-1160

[21] C. Sciavolino, Z. Zhong, J. Lee, and D. Chen, "Simple entity-centric questions challenge dense retrievers," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2021, pp. 6138–6148.

[22] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi, "When not to trust language models: Investigating effectiveness of parametric and non-parametric memories," in Proc. 61st Annu. Meeting Assoc. Comput. Linguist. (ACL), 2023, pp. 9802–9822.

[23] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2016, pp. 2383–2392.

[24] S. Min, J. Michael, H. Hajishirzi, and L. Zettlemoyer, "AmbigQA: Answering ambiguous open-domain questions," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2020, pp. 5783–5797.

[25] O. Press, M. Zhang, S. Min, L. Schmidt, N. Smith, and M. Lewis, "Measuring and narrowing the compositionality gap in language models," in Findings Assoc. Comput. Linguist.: EMNLP, 2023, pp. 5687–5711.

[26] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov et al., "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2018, pp. 2369–2380.

[27] X. Ho, A.-K. Duong Nguyen, S. Sugawara, and A. Aizawa, "Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps," in Proc. 28th Int. Conf. Comput. Linguist. (COLING), 2020, pp. 6609–6625.

[28] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "MuSiQue: Multihop questions via single-hop question composition," Trans. Assoc. Comput. Linguist., vol. 10, pp. 539–554, 2022.

[29] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov et al., "Dense passage retrieval for open-domain question answering," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2020, pp. 6769–6781.